

Projectverslag Computer Graphics 2025-2026

Neys Michael (2467626)

5 augustus 2026

1 Inleiding

Dit verslag beschrijft de implementatie van het Computer Graphics project voor het academiejaar 2025-2026. In onze applicatie hebben we een 3D-omgeving gebouwd met OpenGL (via GLFW en GLAD) waarin een bij (bee) over een traject van samengestelde Bézier-curves vliegt. Dit verslag geeft een gedetailleerd overzicht van de geïmplementeerde functionaliteiten, de gebruikte optimalisatietechnieken en de software-architectuur.

2 Geïmplementeerde Functionaliteiten

2.1 Curve, Animatie en Booglengte-parameterisatie

Het traject van de bij is opgebouwd uit samengestelde Cubic Bézier-splines aan de hand van controlepunten die voldoen aan de $(3n + 1)$ voorwaarde in de `BezierPath` klasse.

Om te garanderen dat de bij met een constante snelheid over de curve beweegt, is een booglengte-parameterisatie geïmplementeerd:

- **Look-Up Table (LUT):** Tijdens de initialisatie bouwt de functie `buildLUT` een tabel op die de booglengte koppelt aan de t -parameter.
- **Binaire Zoekopdracht ($\mathcal{O}(\log N)$):** Om de gewenste afstand om te zetten naar de bijbehorende t -parameter gebruikt `getTForDistance` het C++ `std::lower_bound` algoritme. Dit reduceert de zoektijd in de LUT van een lineaire $\mathcal{O}(N)$ naar een logaritmische $\mathcal{O}(\log N)$ complexiteit.
- **Lineaire Interpolatie:** De exacte t -waarde tussen twee LUT-ijkpunten wordt via lineaire interpolatie berekend. Dit voorkomt schokkerige bewegingen bij lage LUT-resoluties.

2.2 Modellen, Terrein en Traject-rendering

De 3D-wereld is opgebouwd uit modulaire sub-systemen:

- **Voxel Terrein:** Een procedureel gegenereerd hoogte-terrein via de `Terrain` klasse.
- **Skybox:** Een `Skybox` klasse die 6 cubemap-texturen inlaadt via `stb_image`. Tijdens het renderen wordt de translatiematrix uit de View-matrix gestript en wordt de dieptetest ingesteld op `GL_LEQUAL` en `glDepthMask(GL_FALSE)`, zodat de achtergrond altijd achter alle 3D-objecten blijft liggen.
- **TrackRenderer & Pollen:** De `TrackRenderer` genereert via *Forward Differencing* een visuele groene hulplijn op de GPU. Tevens berekent deze klasse willekeurig verdeelde transformatiematrices voor kleine stuifmeelkorrels langs de route.

2.3 Camera's

Er zijn twee camera-modi geïmplementeerd die gewisseld kunnen worden met de 'C'-toets:

- **Free-look/fly camera:** Hiermee kan de hele scène vrij verkend worden.
 - W, A, S, D: Navigatie (vooruit, links, achteruit, rechts).
 - Spatiebalk: Verticaal omhoog bewegen.
 - Control (Ctrl): Verticaal omlaag bewegen.
 - Shift: Versnellen/sprinten in de richting waarin je momenteel beweegt.
- **First-person camera (Bij):** Deze camera is vastgemaakt aan de bij. De View-matrix wordt berekend op basis van de actuele positie op de curve en de richting naar het volgende punt op het traject (`glm::lookAt`).

2.4 Belichting

Per-pixel belichting met point lights waarvan de intensiteit afneemt over afstand:

- **Per-pixel Belichting:** Geïmplementeerd in de fragment shader (`lighting.frag`); ambient, diffuse en specular. De berekening gebeurt per pixel op basis van de genormaliseerde normaal- en kijkvectoren.
- **Lokale Lichtbronnen (18 point lights):** Er zijn 18 lichtbronnen langs het parcours en rond het station geplaatst.
- **Afstandsverzwakking (Attenuation):** De invloed van elke lantaarn neemt realistisch af met de afstand via de formule:

$$\text{Attenuation} = \frac{1.0}{\text{constant} + \text{linear} \cdot d + \text{quadratic} \cdot d^2}$$

- **Lichtbeheer & Optimalisatie:** De `LightManager` klasse beheert de uniform-parameters. Om rendering-overhead te vermijden, worden de uniforme namen vooraf gecached. Met de 'R'-toets kunnen de lampen getoggled worden.

2.5 Convolutie en Post-processing (Bloom)

Post-processing wordt toegepast door de scène te renderen naar een Framebuffer Object (FBO):

- **Convolutie:** Via de toetsen **1, 2 en 3** kan geschakeld worden tussen de standaardweergave, een Gaussian Blur en Edge Detection (Laplaciaan/Sobel).
- **Bloom:** Lantaarns krijgen een realistisch glow-effect via een brightness-extractie shader (`bloom_bright.frag`). Vervolgens wordt deze buffer via 'ping-pong' framebuffers in meerdere passes ge-blurred en samengevoegd met de originele scène. Dit kan getoggled worden via de 'B'-toets.

2.6 GPU Color-based Picking (Interactie)

Voor de interactie met objecten is gekozen voor color-based picking (Off-screen Rendering) in plaats van traditionele CPU-raycasting:

- **Picking FBO & Shader:** Er is een dedicated Framebuffer Object (`pickingFBO`) en een specifieke `pickingShader`.

- **Off-screen Rendering:** Bij een muisklik worden de interactieve meshes (zoals de lamp-mesh) off-screen naar de FBO getekend met een unieke identificatiekleur (puur rood: RGB(255, 0, 0)). De belichting en textures worden hierbij overgeslagen.
- **Pixel Uitlezen (glReadPixels):** De applicatie leest direct de kleurwaarde uit van de enkele pixel in het midden van de buffer (`screenWidth / 2, screenHeight / 2`).
- **Hit Detection:** Als de uitgelezen kleur exact overeenkomt met de rode id-kleur (`pixel[0] == 255`), wordt een succesvolle interactie gedetecteerd. Dit vuurt de `toggleLamp()` en `toggleTrack()` functies af.

2.7 Chroma-keying

Via de `ChromaKey` klasse is een textured quad aan de scène toegevoegd:

- De overlay-textuur wordt ingeladen met `stb_image` en gerenderd met de `chromaKey.frag` shader.
- Chroma-keying detecteert de specifieke groene achtergrondkleur en verwijdert deze met het GLSL `discard` commando.
- Om beeldvervalsing te voorkomen worden de afmetingen van het vlak dynamisch geschaald op basis van de aspect ratio van de afbeelding.

3 Niet-geïmplementeerde Functionaliteiten

Alle basisfunctionaliteiten uit de opgave werden geïmplementeerd.

4 Planning en Werkverdeling

We hebben beide actief meegewerkt aan het project via Git. In de onderstaande tabel wordt de tijdsbesteding (per persoon, per dag/periode en het aantal uren) inzichtelijk gemaakt aan de hand van onze bijgehouden urenregistratie, zoals vereist in de opgave.

Datum	Tijd	Persoon	Taak / Beschrijving
10 april	2 min	Michael	Werkuren sheet maken
10 april	10 min	Maximiliaan	Trello opbouwen
14 - 15 april	10 uur	Maximiliaan	Base template opzetten
14 april	1.5 uur	Michael	Clean up repo & GitHub workflows (Linux/Windows)
19 april	3.5 uur	Michael	Bezier curve wiskunde & display implementatie
23 - 30 april	11 uur	Maximiliaan	Texture displaying
3 - 5 mei	15 uur	Michael	Track door treinstation modelleren/plaatsen
5 mei	1.5 uur	Michael	Bij de track laten volgen (constante snelheid & kijkrichting)
5 - 23 mei	5.5 uur	Michael	Belichting van de lantaarns
6 - 14 mei	7 uur	Maximiliaan	Terrain toevoegen
6 - 14 mei	5 uur	Maximiliaan	Skybox toevoegen
10 mei	1.5 uur	Samen	Call voor overleg
14 mei	1 uur	Maximiliaan	Meer controls voor free-camera
17 - 23 mei	15 uur	Maximiliaan	Convolutie algemeen opzetten & toevoegen
17 - 23 mei	3 uur	Maximiliaan	Gaussian Blur
17 - 23 mei	5 uur	Maximiliaan	Laplaciaan kernel
23 - 24 mei	7 uur	Maximiliaan	Bloom implementatie
23 - 24 mei	3 uur	Maximiliaan	First-person camera bij
23 - 24 mei	2 uur	Samen	Verslag schrijven
24 mei	2.5 uur	Michael	Picking voor interactie (raycasting)
24 mei	30 min	Michael	Resizing van windows correct afhandelen
24 mei	15 min	Michael	Cursor popout (mouse capture met ESC)
24 mei	3 uur	Michael	Opschonen van de code
24 mei	1 uur	Maximiliaan	Chroma-keying
24 mei	1 uur	Michael	Windows build werkende krijgen

Tabel 1: Overzicht van de daadwerkelijk gevolgde planning, taakverdeling en bestede tijd.